

ESTUDO DIRIGIDO – VETORES E LISTAS

Disciplina: Linguagem de Programação

Prof. Edson Rodrigues de Aguiar

Orientações

Resolva as questões conceituais com suas próprias palavras. Nos exercícios de programação, desenvolva o algoritmo e, quando solicitado, implemente a solução em C++.

Parte I – Questões conceituais

1. O que é uma estrutura de dados? Dê um exemplo de sua utilização em um programa.
 - Estrutura de dados é uma forma organizada de armazenar, gerenciar e acessar informações na memória do computador para que possam ser usadas com eficiência. Ele segue o princípio FIFO (First In, First Out). Uma fila de banca ou a fila de atendimento, por exemplo, pode ser usada para ilustrar a estrutura de dados no nosso cotidiano.
2. Explique com suas palavras o que é um vetor e como seus elementos são organizados.
 - Um vetor, ou chamado também de array, é uma estrutura de dados que armazena uma coleção de elementos do mesmo tipo sob um único nome. São organizados de forma contínua e seu acesso é por índices, cada posição do vetor possui um número identificador chamado índice que tem seu início no 0.
3. Considere o vetor: `int valores[5] = {4, 8, 12, 16, 20};`
 - a) Qual é o valor de `valores[0]`?
 - 4
 - b) Qual é o valor de `valores[3]`?
 - 16
 - c) Em qual índice está armazenado o número 20?
 - [4]
 - d) Quantos elementos existem no vetor?
 - Existem 5 valores numéricos
4. Qual é a principal diferença entre um vetor e uma lista encadeada?
 - Um vetor tem elementos identificados por índice, desta forma, possibilita retirar, adicionar ou modificar um valor chamando-o pelo seu índice, independente de sua posição. Já na lista, só é possível remover ou adicionar o primeiro que entrou.
5. O que é um nó em uma lista encadeada? Quais informações normalmente fazem parte de um nó?

- É a unidade básica de construção de uma lista encadeada, funcionando como um pequeno bloco de armazenamento independente na memória. As informações do nó são: campo de dados (valor) e campo de referência (ponteiro).

6. Qual é a função do ponteiro próximo em uma lista simplesmente encadeada?

- A função do ponteiro ‘próximo’ em uma lista simplesmente encadeada é armazenar o endereço de memória do próximo nó da sequência. Ele liga um elemento ao seguinte, permitindo percorrer a lista de forma encadeada.

7. Por que o último nó de uma lista encadeada aponta para NULL ou nullptr?

- O último nó de uma lista encadeada aponta para null (ou nullptr) para sinalizar o fim da estrutura, indicando que não existe um próximo elemento. Evita que o programa tente acessar um endereço de memória inválido ou lixo de memória, prevenindo erros graves de execução.

8. Cite duas vantagens do vetor e duas vantagens da lista encadeada.

- O vetor possibilita: acesso direto, permite encontrar qualquer item de forma instantânea usando a sua posição numérica (índice); e uso eficiente de memória, armazena os dados lado a lado na memória sem precisar de espaço extra para apontadores. A lista encadeada possibilita: ter um tamanho flexível que cresce ou diminui conforme adiciona ou retira itens, sem precisar definir um limite inicial; inserção e remoção rápidas que permitir incluir ou apagar elementos no meio da estrutura sem precisar reorganizar o resto dos dados.

Parte II – Exercícios com Vetores

9. Preenchimento e exibição

Crie um programa em C++ que receba 10 números inteiros, armazene-os em um vetor e depois apresente todos os valores armazenados.

Espaço para algoritmo/código:

```
#include <iostream>
using namespace std;
int main (){
    int vetor[10];
    cout << "Digite 10 numeros inteiros: \n";
    for (int i=0; i<10; i++) {
        cout<< "Numero " << (i + 1) << ": ";
        cin >> vetor[i];
    }
    cout << "\n Valores armazenados no vetor:\n";
    for (int i = 0; i< 10; i++) {
        cout << vetor[i] << " ";
    }
    cout << endl;
```

```
        return 0;
    }
```

10. Maior e menor valor

Utilizando um vetor com 10 números inteiros, determine o maior valor, o menor valor, a posição do maior valor e a posição do menor valor.

Espaço para algoritmo/código:

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> numeros = {15, 42, 7, 89, 23, 56, 2, 71, 38, 12};
    int maior = numeros[0];
    int menor = numeros[0];
    int pos_maior = 0;
    int pos_menor = 0;
    for (size_t i = 1; i < numeros.size(); ++i) {
        if (numeros[i] > maior) {
            maior = numeros[i];
            pos_maior = i;
        }
        if (numeros[i] < menor) {
            menor = numeros[i];
            pos_menor = i;
        }
    }
    cout << "Maior valor: " << maior << " na posicao: " << pos_maior << endl;
    cout << "Menor valor: " << menor << " na posicao: " << pos_menor << endl;
    return 0;
}
```

11. Média das notas

Desenvolva um programa que armazene as notas de 5 alunos em um vetor e apresente: todas as notas, a média da turma, a maior nota, a menor nota e a quantidade de notas acima da média.

Espaço para algoritmo/código:

```
#include <iostream>
#include <iomanip> // Para formatar as casas decimais

using namespace std;

int main() {
    const int totalAlunos = 5;
    float notas[totalAlunos];
    float soma = 0.0, media = 0.0;
    float maior, menor;
```

```

int acimaDaMedia = 0;

// Leitura das notas
cout << "Digite as notas dos 5 alunos:" << endl;
for (int i = 0; i < totalAlunos; i++) {
    cout << "Aluno " << (i + 1) << ": ";
    cin >> notas[i];
    soma += notas[i];
}

// Cálculo da média
media = soma / totalAlunos;

// Inicialização da maior e menor nota com o primeiro elemento
maior = notas[0];
menor = notas[0];

// Processamento para encontrar maior, menor e quantidade acima da média
for (int i = 0; i < totalAlunos; i++) {
    if (notas[i] > maior) {
        maior = notas[i];
    }
    if (notas[i] < menor) {
        menor = notas[i];
    }
    if (notas[i] > media) {
        acimaDaMedia++;
    }
}

// Exibição dos resultados com 2 casas decimais
cout << fixed << setprecision(2);
cout << "\n--- RESULTADOS ---" << endl;

cout << "Notas cadastradas: ";
for (int i = 0; i < totalAlunos; i++) {
    cout << notas[i] << " ";
}
cout << endl;

cout << "Média da turma: " << media << endl;
cout << "Maior nota: " << maior << endl;
cout << "Menor nota: " << menor << endl;
cout << "Quantidade de notas acima da média: " << acimaDaMedia << endl;

return 0;
}

```

12. Pares e ímpares

Leia 10 números inteiros e armazene-os em um vetor. Ao final, informe a quantidade de números pares e a quantidade de números ímpares.

Espaço para algoritmo/código:

```
#include <iostream>

using namespace std;

int main() {
    int vetor[10];
    int pares = 0;
    int impares = 0;

    // Leitura dos 10 números inteiros
    cout << "Digite 10 números inteiros:" << endl;
    for (int i = 0; i < 10; i++) {
        cout << "Número " << i + 1 << ": ";
        cin >> vetor[i];
    }

    // Verificação de pares e ímpares usando o vetor
    for (int i = 0; i < 10; i++) {
        if (vetor[i] % 2 == 0) {
            pares++;
        } else {
            impares++;
        }
    }

    // Exibição dos resultados
    cout << "\nQuantidade de números pares: " << pares << endl;
    cout << "Quantidade de números ímpares: " << impares << endl;

    return 0;
}
```

13. Busca em vetor

Leia 10 números e depois solicite ao usuário um número para procurar. Caso o valor seja encontrado, informe sua posição. Caso contrário, apresente a mensagem “Valor não encontrado”.

Espaço para algoritmo/código:

```
#include <iostream>
```

```

using namespace std;

int main() {
    int numeros[10];
    int numeroProcurado;
    bool encontrado = false;

    // Leitura dos 10 números
    cout << "Digite 10 numeros:" << endl;
    for (int i = 0; i < 10; i++) {
        cout << "Numero " << i + 1 << ": ";
        cin >> numeros[i];
    }

    // Solicita o número para busca
    cout << "\nDigite um numero para procurar: ";
    cin >> numeroProcurado;

    // Busca pelo valor no vetor
    for (int i = 0; i < 10; i++) {
        if (numeros[i] == numeroProcurado) {
            cout << "Valor encontrado na posicao: " << i + 1 << endl;
            encontrado = true;
            break; // Interrompe a busca ao encontrar a primeira ocorrência
        }
    }

    // Caso não tenha encontrado
    if (!encontrado) {
        cout << "Valor nao encontrado" << endl;
    }

    return 0;
}

```

Parte III – Exercícios com Listas Encadeadas

14. Identificando os elementos

Considere a lista: $10 \rightarrow 25 \rightarrow 30 \rightarrow 45 \rightarrow \text{NULL}$. Responda:

- a) Qual é o primeiro elemento?
 - 10
- b) Qual é o último elemento?
 - 45
- c) Qual elemento vem depois do 25?
 - 30
- d) Quantos nós existem?
 - 4

e) Para onde aponta o nó que contém o valor 45?

- para o NULL

Resposta / código:

15. Inserção em uma lista

Considere a lista $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$. Represente graficamente como ela ficará depois de inserir o valor 5 no início. Em seguida, represente a inserção do valor 40 no final.

Resposta / código:

- $5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{NULL}$

16. Remoção em uma lista

Considere a lista $5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{NULL}$. Mostre como ficará após remover, nesta ordem:

a) 5; b) 30; c) 40.

Resposta / código:

- $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{NULL}$; $10 \rightarrow 20 \rightarrow 40 \rightarrow \text{NULL}$; $10 \rightarrow 20 \rightarrow \text{NULL}$

17. Inserção no início em C++

Utilizando a estrutura abaixo, desenvolva uma função para inserir um elemento no início da lista:

```
struct No {  
    int valor;  
    No* proximo;  
};
```

Protótipo sugerido: `void inserirInicio(No*& inicio, int valor);`

Resposta / código:

```
#include <iostream>
```

```
struct No {  
    int valor;  
    No* proximo;  
};  
void inserirInicio(No*& inicio, int valor) {  
    No* novo = new No();  
    novo->valor = valor;  
    novo->proximo = inicio;  
    inicio = novo;  
}
```

18. Percorrendo uma lista

Desenvolva uma função em C++ que percorra uma lista encadeada e apresente todos os seus elementos.

Resposta / código:

```
#include <iostream>

using namespace std;

struct No {
    int dado;
    No* proximo;
};

void exibirLista(No* cabeca) {
    No* atual = cabeca;
    if (atual == nullptr) {
        cout << "A lista esta vazia." << endl;
        return;
    }
    while (atual != nullptr) {
        cout << atual->dado << " -> ";
        atual = atual->proximo;
    }
    cout << "NULL" << endl;
}

int main () {
    cout << "Testando";
    return 0;
}
```

19. Busca em lista

Implemente a função bool buscar(No* inicio, int valor);. Ela deverá retornar true quando o elemento existir na lista e false quando não existir.

Resposta / código:

```
#include <iostream>
using namespace std;
// Estrutura do nó da lista (exemplo)
struct No {
    int valor;
    No* proximo;
};
// Implementação da função de busca
bool buscar(No* inicio, int valor) {
    No* atual = inicio;
    while (atual != nullptr) {
        if (atual->valor == valor) {
            return true; // Elemento encontrado
        }
        atual = atual->proximo; // Avança para o próximo nó
    }
}
```

```
    return false; // Elemento não existe na lista
}
```

Parte IV – Desafios

20. Sistema de notas com vetor

Uma turma possui 10 alunos. Desenvolva um programa que permita: cadastrar as 10 notas; apresentar todas as notas; calcular a média; apresentar a maior e a menor nota; informar quantos alunos ficaram acima da média; e permitir procurar uma determinada nota.

[Escreva aqui o código]

```
#include <iostream>
#include <vector>

using namespace std;

void cadastrarNotas(vector<double>& notas) {
    cout << "--- Cadastro de Notas ---" << endl;
    for (int i = 0; i < 10; i++) {
        cout << "Digite a nota do aluno " << i + 1 << ": ";
        cin >> notas[i];
    }
}

void exibirNotas(const vector<double>& notas) {
    cout << "\n--- Notas Cadastradas ---" << endl;
    for (int i = 0; i < 10; i++) {
        cout << "Aluno " << i + 1 << ": " << notas[i] << endl;
    }
}

double calcularMedia(const vector<double>& notas) {
    double soma = 0;
    for (double nota : notas) {
        soma += nota;
    }
    return soma / 10;
}

void exibirMaiorMenor(const vector<double>& notas) {
    double maior = notas[0];
    double menor = notas[0];

    for (int i = 1; i < 10; i++) {
        if (notas[i] > maior) maior = notas[i];
        if (notas[i] < menor) menor = notas[i];
    }
}
```

```

    }
    cout << "Maior nota: " << maior << endl;
    cout << "Menor nota: " << menor << endl;
}

void alunosAcimaDaMedia(const vector<double>& notas, double media) {
    int contador = 0;
    for (double nota : notas) {
        if (nota > media) {
            contador++;
        }
    }
    cout << "Quantidade de alunos acima da média (" << media << "): " << contador << endl;
}

void procurarNota(const vector<double>& notas) {
    double notaBusca;
    cout << "\nDigite uma nota para procurar: ";
    cin >> notaBusca;

    bool encontrada = false;
    for (int i = 0; i < 10; i++) {
        if (notas[i] == notaBusca) {
            cout << "Nota " << notaBusca << " encontrada no Aluno " << i + 1 << endl;
            encontrada = true;
        }
    }

    if (!encontrada) {
        cout << "A nota " << notaBusca << " nao foi encontrada na turma." << endl;
    }
}

int main() {
    vector<double> notas(10);

    cadastrarNotas(notas);
    exibirNotas(notas);

    cout << "\n--- Estatísticas ---" << endl;
    double media = calcularMedia(notas);
    cout << "Média da turma: " << media << endl;

    exibirMaiorMenor(notas);
    alunosAcimaDaMedia(notas, media);

    procurarNota(notas);

    return 0;
}

```

21. Cadastro utilizando lista encadeada

Desenvolva um programa utilizando lista encadeada que apresente o menu abaixo e execute as operações correspondentes:

- 1 - Inserir elemento
- 2 - Listar elementos
- 3 - Buscar elemento
- 4 - Remover elemento
- 5 - Contar elementos
- 0 - Sair

```
#include <iostream>

using namespace std;

struct Nodo {
    int dado;
    Nodo* proximo;
};

void inserirElemento(Nodo*& topo, int valor) {
    Nodo* novoNodo = new Nodo();
    novoNodo->dado = valor;
    novoNodo->proximo = topo;
    topo = novoNodo;
    cout << "Elemento " << valor << " inserido com sucesso!\n";
}

void listarElementos(Nodo* topo) {
    if (topo == nullptr) {
        cout << "A lista esta vazia.\n";
        return;
    }

    Nodo* atual = topo;
    cout << "Elementos da lista: ";
    while (atual != nullptr) {
        cout << atual->dado << " -> ";
        atual = atual->proximo;
    }
    cout << "NULL\n";
}

bool buscarElemento(Nodo* topo, int valor) {
```

```

    Nodo* atual = topo;
    while (atual != nullptr) {
        if (atual->dado == valor) {
            return true;
        }
        atual = atual->proximo;
    }
    return false;
}

```

```

void removerElemento(Nodo*& topo, int valor) {
    if (topo == nullptr) {
        cout << "A lista esta vazia. Nao ha o que remover.\n";
        return;
    }

```

```

    Nodo* atual = topo;
    Nodo* anterior = nullptr;

```

```

    if (atual != nullptr && atual->dado == valor) {
        topo = atual->proximo;
        delete atual;
        cout << "Elemento " << valor << " removido com sucesso!\n";
        return;
    }

```

```

    while (atual != nullptr && atual->dado != valor) {
        anterior = atual;
        atual = atual->proximo;
    }

```

```

    if (atual == nullptr) {
        cout << "Elemento " << valor << " nao encontrado na lista.\n";
        return;
    }

```

```

    anterior->proximo = atual->proximo;
    delete atual;
    cout << "Elemento " << valor << " removido com sucesso!\n";
}

```

```

int contarElementos(Nodo* topo) {
    int contador = 0;
    Nodo* atual = topo;
    while (atual != nullptr) {
        contador++;
        atual = atual->proximo;
    }
    return contador;
}

```

```

int main() {
    Nodo* lista = nullptr;
    int opcao, valor;

    do {
        cout << "\n=== MENU DE OPCOES ===\n";
        cout << "1 - Inserir elemento\n";
        cout << "2 - Listar elementos\n";
        cout << "3 - Buscar elemento\n";
        cout << "4 - Remover elemento\n";
        cout << "5 - Contar elementos\n";
        cout << "0 - Sair\n";
        cout << "Escolha uma opcao: ";
        cin >> opcao;

        switch (opcao) {
            case 1:
                cout << "Digite o valor para inserir: ";
                cin >> valor;
                inserirElemento(lista, valor);
                break;
            case 2:
                listarElementos(lista);
                break;
            case 3:
                cout << "Digite o valor para buscar: ";
                cin >> valor;
                if (buscarElemento(lista, valor)) {
                    cout << "Elemento " << valor << " encontrado na lista.\n";
                } else {
                    cout << "Elemento " << valor << " nao encontrado.\n";
                }
                break;
            case 4:
                cout << "Digite o valor para remover: ";
                cin >> valor;
                removerElemento(lista, valor);
                break;
            case 5:
                cout << "Total de elementos na lista: " << contarElementos(lista) << "\n";
                break;
            case 0:
                cout << "Saindo do programa...\n";
                break;
            default:
                cout << "Opcao invalida! Tente novamente.\n";
        }
    } while (opcao != 0);

    while (lista != nullptr) {
        Nodo* temp = lista;
    }
}

```

```

    lista = lista->proximo;
    delete temp;
}

return 0;
}

```

Parte V – Questão de análise

22. Uma instituição precisa armazenar uma quantidade de registros que muda constantemente. Durante a execução do programa, novos registros são adicionados e registros antigos podem ser removidos. Compare VETOR e LISTA ENCADEADA e explique qual estrutura seria mais adequada considerando especificamente inserções/remoções e acesso aos elementos. Cite pelo menos uma vantagem e uma desvantagem de cada estrutura.

- A lista encadeada é mais adequada, pois a quantidade de registros varia constantemente. Ela facilita a inserção e remoção de elementos. Porém, seu acesso aos elementos é mais lento que o de um vetor. O vetor possui acesso rápido por índice, mas inserções e remoções podem exigir deslocamento de elementos e redimensionamento.

ro para síntese

Característica	Vetor	Lista Encadeada
Tamanho	Fixo ou precisa ser redimensionado	Dinâmico, cresce e diminui conforme necessário
Acesso aos elementos	Rápido e direto, por índice	Mais lento, pois é necessário percorrer a lista
Inserção/remoção	Mais lenta, principalmente no início ou meio	Mais rápida, pois basta ajustar os ponteiros
Uso de memória	Usa memória contínua; pode haver espaço desperdiçado	Usa memória conforme os nós são criados, mas cada nó precisa armazenar ponteiros
Exemplo de aplicação	Tabelas, notas, dados que precisam de acesso rápido por índice	Filas, listas de registros que mudam frequentemente

